

WRITE_TO_PERSISTENCE

Method Deep Dive — ZCL_PO_LONGTEXT_HANDLER

Every parameter explained | 5-step internal flow | All field meanings | 2026-06-28

One-Line Summary

"Read compressed text from SAP → decompress it → delete old version → store each line as plain readable row in ZPO_LONGTEXT."

Why This Method Exists

SAP stores long texts in STXL in compressed binary format. You cannot read STXL with a plain SELECT — it is unreadable. Every time you need the text you must call READ_TEXT to decompress it. This is slow, cannot be used in CDS views, and cannot be used in BDC programs easily. WRITE_TO_PERSISTENCE solves this by decompressing ONCE and storing the result as plain rows in ZPO_LONGTEXT where any program can SELECT directly.

Without ZPO_LONGTEXT	With ZPO_LONGTEXT (after WRITE_TO_PERSISTENCE)
CALL FUNCTION 'READ_TEXT' for every read — slow at scale	SELECT * FROM zpo_longtext WHERE ebeln = '...' — instant
STXL is compressed binary — cannot use in CDS view	TDLINE is plain text — fully usable in CDS, reports, BDC
Cannot join PO text with other tables in SQL	Can join ZPO_LONGTEXT with EKKO, EKPO etc. in one SELECT
READ_TEXT fails in background jobs silently	Plain SQL always works in any context

P

All 5 Parameters — Complete Reference

Every parameter with type, default, allowed values and purpose

IV_EBELN

Type: EBELN (CHAR 10)

Default: —

Mandatory

Purchase Order Number Example: '4500077744' Used for: (1) build TDNAME for READ_TEXT call (2) stored as key in ZPO_LONGTEXT EBELN field

IV_EBELP

Type: EBELP (NUMC 5)

Default: '00000'

Optional

PO Item Number Default '00000' = header text (TDOBJECT=EKKO) Pass explicitly for item texts: '00010', '00020' etc. Header call: omit EBELP → defaults to 00000 automatically Item call: must pass: iv_ebelp = '00010'

IV_TDOBJECT

Type: TDOBJECT (CHAR 10)

Default: —

Mandatory

Text Object — identifies the SAP business object EKKO = Purchase Order Header text EKPO = Purchase Order Item text Controls TDNAME construction: EKKO → TDNAME = EBELN only (e.g. 4500077744) EKPO → TDNAME = EBELN+EBELP no space (e.g. 450007774400010)

IV_TDID

Type: TDID (CHAR 4)

Default: —

Mandatory

Text ID — identifies which type of text F01 = General Note / Header Note ← confirmed this system F02 = Delivery Text ← confirmed this system F09 = Goods Receipt Text F03, F04, F05... other text types One PO can have F01 AND F02 — each requires a separate call

IV_TDSPRAS

Type: TDSRAS (LANG 1)

Default: —

Mandatory

Language Key D = German ← confirmed this system E = English F = French etc. In BADl: pass sy-langu (logged-in user language) Same PO text can exist in multiple languages — each language = separate rows

IV_EBELP defaults to '00000' — this means for HEADER texts you can omit it: write_to_persistence(iv_ebeln='4500077744' iv_tdoobject='EKKO' iv_tdid='F01' iv_tdspras='D') ← IV_EBELP not passed → automatically = 00000 For ITEM texts you MUST pass IV_EBELP explicitly: write_to_persistence(iv_ebeln='4500077744' iv_ebelp='00010' iv_tdoobject='EKPO' iv_tdid='F01' iv_tdspras='D')

5

5-Step Internal Flow

What happens inside `WRITE_TO_PERSISTENCE` when you call it

1

Build TDNAME

Construct the key that `READ_TEXT` needs

`READ_TEXT` requires a field called `TDNAME` — the document key for the text. It is built differently for header vs item:

```
IF iv_tdoobject = 'EKKO'.           ← Header text

    lv_tdname = iv_ebeln.           ← TDNAME = PO number only

    " Example: lv_tdname = '4500077744'

ELSE.                               ← Item text (EKPO)

    CONCATENATE iv_ebeln iv_ebelp INTO lv_tdname.

    " Example: lv_tdname = '450007774400010' (NO space – confirmed this system)

ENDIF.
```

Text Level	TDOBJECT	TDNAME Format	Example
Header	EKKO	EBELN only	4500077744
Item	EKPO	EBELN + EBELP (no space)	450007774400010

■ `TDNAME` for `EKPO` has NO SPACE on this system (confirmed from `STXH`). Standard SAP documentation says `EBELN + SPACE + EBELP` — NOT valid here.

Call READ_TEXT

2

Decompress text from STXL — the only way to read it

READ_TEXT is SAP's standard function module that reads and decompresses text stored in STXL. It returns a plain TLINE table.

```
CALL FUNCTION 'READ_TEXT'

EXPORTING

  id      = iv_tdid      ← F01, F02 etc.

  language = iv_tdspras  ← D, E etc.

  name     = lv_tlname   ← built in Step 1

  object   = iv_tdoobject ← EKKO or EKPO

TABLES

  lines    = lt_lines    ← returns plain text lines here

EXCEPTIONS

  OTHERS   = 4.          ← sy-subrc = 4 if text not found
```

What READ_TEXT does internally:

- Looks up STXH for matching TDOBJECT + TDID + TDSRAS + TDNAME
- Gets the STXL cluster row (binary compressed data)
- Decompresses the binary → returns plain TLINE table

lt_lines result (example – 2 lines of text):

```
TDFORMAT  TDLINE

*          PO header note line 1    ← * = normal text line

*          PO header note line 2
```

Check if Text Exists

3

Exit cleanly if no text found

```
IF sy-subrc <> 0 OR lt_lines IS INITIAL.  
  
    RETURN.    ← exit — nothing to write  
  
ENDIF.
```

When does this happen?

```
sy-subrc <> 0 → READ_TEXT failed:  
  
    PO has no text for this TDID  
  
    Wrong TDNAME was constructed  
  
lt_lines IS INITIAL → Text header exists in STXH but content is empty
```

■ This RETURN is safe — it means no text exists for this combination. The method exits without writing anything to ZPO_LONGTEXT. No error, no dump — just a clean exit.

Delete Old Rows

4

Full refresh — remove previous version before inserting new

```
DELETE FROM zpo_longtext  
  
WHERE ebeln      = iv_ebeln  
  
    AND ebelp      = iv_ebelp  
  
    AND tdoobject = iv_tdoobject  
  
    AND tdid       = iv_tdid  
  
    AND tdspras    = iv_tdspras.
```

Why DELETE before INSERT — the stale data problem:

Scenario	Without DELETE	With DELETE + INSERT
User had 3 text lines, reduced to 2 in ME22N	Old line 3 stays in ZPO_LONGTEXT — WRONG data	All 3 old lines deleted → 2 fresh lines inserted — CORRECT
User changed text content	Old content + new content coexist — DUPLICATE	Old content deleted → new content only — CORRECT
User deleted the text entirely	Stale rows remain forever — GHOST DATA	All rows deleted → nothing inserted (Step 3 returns) — CORRECT

Insert Plain Rows

5

One row per text line into ZPO_LONGTEXT

```
LOOP AT lt_lines INTO DATA(ls_line).

  ADD 1 TO lv_counter.          ← 00001, 00002, 00003...

  CLEAR ls_persist.

  ls_persist-ebeln      = iv_ebeln.      ← PO number
  ls_persist-ebelp      = iv_ebelp.      ← 00000 or item#
  ls_persist-tdobject   = iv_tdobject.   ← EKK0 or EKPO
  ls_persist-tdid       = iv_tdid.       ← F01, F02 etc.
  ls_persist-tdspras    = iv_tdspras.    ← D
  ls_persist-line_counter = lv_counter.  ← sequence number
  ls_persist-tdformat   = ls_line-tdformat. ← * = normal line
  ls_persist-tdline     = ls_line-tdline. ← PLAIN TEXT content
  ls_persist-migrated_at = sy-datum.     ← today's date
  ls_persist-migrated_by = sy-uname.     ← current user
  ls_persist-source     = 'D'.           ← D = Dual-write by BAdI

  INSERT zpo_longtext FROM ls_persist.

ENDLOOP.
```

F

ZPO_LONGTEXT Field Meanings

What every field written by `WRITE_TO_PERSISTENCE` contains

Field	Set From	Example	Meaning
EBELN	iv_ebeln	4500077744	PO number — links this row to the Purchase Order
EBELP	iv_ebelp	00000 / 00010	00000 = header text. 00010 = item 10 text. Drives header vs item distinction
TDOBJECT	iv_tdoobject	EKKO / EKPO	EKKO = header, EKPO = item. Used in WHERE clauses to filter text level
TDID	iv_tdid	F01 / F02	Which text type. F01=General Note, F02=Delivery Text, F09=GR Text
TDSPRAS	iv_tdspras	D	Language key. Allows same PO to have texts in multiple languages
LINE_COUNTER	lv_counter	00001 / 00002	Sequence within this text block. ORDER BY always uses this field
TDFORMAT	ls_line-tdformat	*	* = normal text line (what user typed). /: = SAPscript command (not user text)
TDLINE	ls_line-tdline	PO header note line 1	THE MAIN FIELD — actual plain text content, max 132 chars. Human-readable.
MIGRATED_AT	sy-datum	20260628	Date this row was written — audit trail for the persistence operation
MIGRATED_BY	sy-uname	SUNDARAMURTS	User who triggered the write — from BAdI save or manual migration run
SOURCE	hard-coded 'D'	D	D = Dual-write by BAdI. M = written by migration report. Useful for debugging

SOURCE Field — D vs M

Value	Full Meaning	When Written	How to Check
D	Dual-write	Automatically by BAdI <code>ZCL_PO_TEXT_BADI_IMPL</code> on every ME21N/ME22N save	SE16N → ZPO_LONGTEXT → SOURCE = D → new POs
M	Migration	By migration report <code>ZTEST_PO_LONGTEXT_MIGRATE</code> run manually for historic POs	SE16N → ZPO_LONGTEXT → SOURCE = M → historic POs

LINE_COUNTER — Why It Matters

Without `LINE_COUNTER` you cannot guarantee text line order.

Example — 3 lines of text in `ZPO_LONGTEXT`:

```
LINE_COUNTER=00001  TDLINE='First line of the header note'

LINE_COUNTER=00002  TDLINE='Second line of the header note'

LINE_COUNTER=00003  TDLINE='Third line of the header note'
```

SELECT always includes: `ORDER BY tdoobject, tdid, tdspras, line_counter`

This guarantees lines come back in correct sequence to reconstruct the text.

If `LINE_COUNTER` had gaps (e.g. 00001, 00003 — missing 00002):

→ Indicates a partial write failure during migration

→ Re-run `WRITE_TO_PERSISTENCE` for that PO to fix

TDFORMAT = * — What It Means

TDFORMAT Value	Meaning	In ZPO_LONGTEXT
*	Normal text line — what the user typed in ME21N	TDLINE = readable text content
/:	SAPscript formatting command line	TDLINE = formatting instruction — not user text
/E	End of text marker	Rarely appears in PO texts

■ Almost all ZPO_LONGTEXT rows will have TDFORMAT = * This means TDLINE contains the actual readable text the user typed. When reading ZPO_LONGTEXT in BDC or reports, you can typically ignore TDFORMAT and just use TDLINE directly for display.

Example 1 — Write Header General Note (F01)

```
" Header text – IV_EBELP omitted (defaults to 00000)

zcl_po_longtext_handler=>write_to_persistence(

    iv_ebeln    = '4500077744'

    iv_tdoobject = 'EKK0'

    iv_tdid     = 'F01'

    iv_tdspras  = 'D' ).
```

Example 2 — Write Header Delivery Text (F02)

```
" Different TDID – separate call for each text type

zcl_po_longtext_handler=>write_to_persistence(

    iv_ebeln    = '4500077744'

    iv_ebelp    = '00000'    " explicit – same as default

    iv_tdoobject = 'EKK0'

    iv_tdid     = 'F02'

    iv_tdspras  = 'D' ).
```

Example 3 — Write Item 10 General Note (EKPO/F01)

```
" Item text – IV_EBELP MUST be passed explicitly

zcl_po_longtext_handler=>write_to_persistence(

    iv_ebeln    = '4500077744'

    iv_ebelp    = '00010'    " ← item number – mandatory for EKPO

    iv_tdoobject = 'EKPO'

    iv_tdid     = 'F01'

    iv_tdspras  = 'D' ).
```

Example 4 — Write All Texts for One PO (Migration Pattern)

```
" Read all STXH entries for this PO and migrate each one

DATA lv_ebeln TYPE ebeln.

DATA lt_stxh  TYPE TABLE OF stxh.

DATA lv_ebelp TYPE ebelp.

DATA lv_tlname_lo TYPE thead-tdname.

DATA lv_tlname_hi TYPE thead-tdname.
```

```

lv_ebeln = '4500077744'.

" Header texts

SELECT tdoobject, tddid, tdsprasad FROM stxh

    INTO CORRESPONDING FIELDS OF TABLE @lt_stxh

    WHERE tdoobject = 'EKK0' AND tddname = @lv_ebeln.

" Item texts (all items)

CONCATENATE lv_ebeln '00000' INTO lv_tddname_lo.

CONCATENATE lv_ebeln '99999' INTO lv_tddname_hi.

SELECT tdoobject, tddid, tdsprasad, tddname FROM stxh

    APPENDING CORRESPONDING FIELDS OF TABLE @lt_stxh

    WHERE tdoobject = 'EKK0'

        AND tddname >= @lv_tddname_lo AND tddname <= @lv_tddname_hi.

LOOP AT lt_stxh INTO DATA(ls_stxh).

    IF ls_stxh-tdoobject = 'EKK0'.

        lv_ebelp = '00000'.

    ELSE.

        lv_ebelp = ls_stxh-tddname+10(5). " parse item from TDNAME

    ENDIF.

zcl_po_longtext_handler=>write_to_persistence(

    iv_ebeln      = lv_ebeln

    iv_ebelp      = lv_ebelp

    iv_tdoobject  = ls_stxh-tdoobject

    iv_tddid      = ls_stxh-tddid

    iv_tdsprasad  = ls_stxh-tdsprasad ).

ENDLOOP.

```

Example 5 — Read Back from ZPO_LONGTEXT

```

DATA lt_texts TYPE ztt_po_longtext.

```

```
zcl_po_longtext_handler=>read_from_persistence(  
  
    EXPORTING iv_ebeln = '4500077744'  
  
    IMPORTING et_lines = lt_texts ).  
  
LOOP AT lt_texts INTO DATA(ls_text).  
  
    WRITE: / ls_text-tdobject, ls_text-tdid,  
           'EBELP:', ls_text-ebelp,  
           'Line:', ls_text-line_counter,  
           ls_text-tdline.  
  
ENDLOOP.  
  
" Output:  
" EKKO F01 EBELP: 00000 Line: 00001 PO header note line 1  
" EKKO F01 EBELP: 00000 Line: 00002 PO header note line 2  
" EKPO F01 EBELP: 00010 Line: 00001 Item 10 note line 1
```